WEEK 02

Relational Algebra and SQL Review Studio

Recover forgotten SQL by predicting result grain, tuples, and evidence first.

02

A correct query begins before the word SELECT

Question → grain → operations → prediction → SQL → verification

BEFORE

Name the source relations and what one result row should mean.

DURING

Translate relational operations into clauses and join conditions.

AFTER

Compare identifiers, counts, nulls, and a second reasoning path.

Result grain controls how every clause should be read

One row per ticket

Selecting ticket attributes keeps the source grain unless a join multiplies rows.

Check ticket identifiers.

One row per event

Joining events to tickets produces repeated ticket values when several events match.

Count events, not tickets.

One row per status

Grouping changes grain from source rows to one result row for each status value.

Check group keys.

Relational operations map to SQL, but semantics still matter

Operation	Relational idea	Common SQL form	Check
Selection σ	keep tuples satisfying a predicate	WHERE	NULL and predicate scope
Projection π	keep named attributes	SELECT columns	SQL keeps duplicates by default
Join ⋈	keep matching tuple pairs	JOIN ... ON	relationship and row multiplication
Grouping γ	form groups and summarize	GROUP BY + aggregates	new result grain
Difference –	tuples in A but not B	EXCEPT / NOT EXISTS	duplicate and NULL behavior

Course-authored relational-to-SQL mapping; see Module 2 for definitions.

Selection and projection become WHERE and the column list

```
-- Grain: one row per matching ticket
SELECT ticket_id, subject, priority
FROM tickets
WHERE priority = 'high'
ORDER BY ticket_id;
```

Predict before running

- Which ticket identifiers should appear?
- Which three attributes remain?
- Does ordering change membership?

NULL means unknown or absent, not an ordinary value

Wrong mental model

- `assignee_id = NULL`
- `status <> 'closed'` includes every other row
- Unknown behaves like false in every context

SQL behavior to test

- Use `IS NULL` or `IS NOT NULL`
- Comparisons with NULL evaluate to unknown
- WHERE retains only predicates that are true

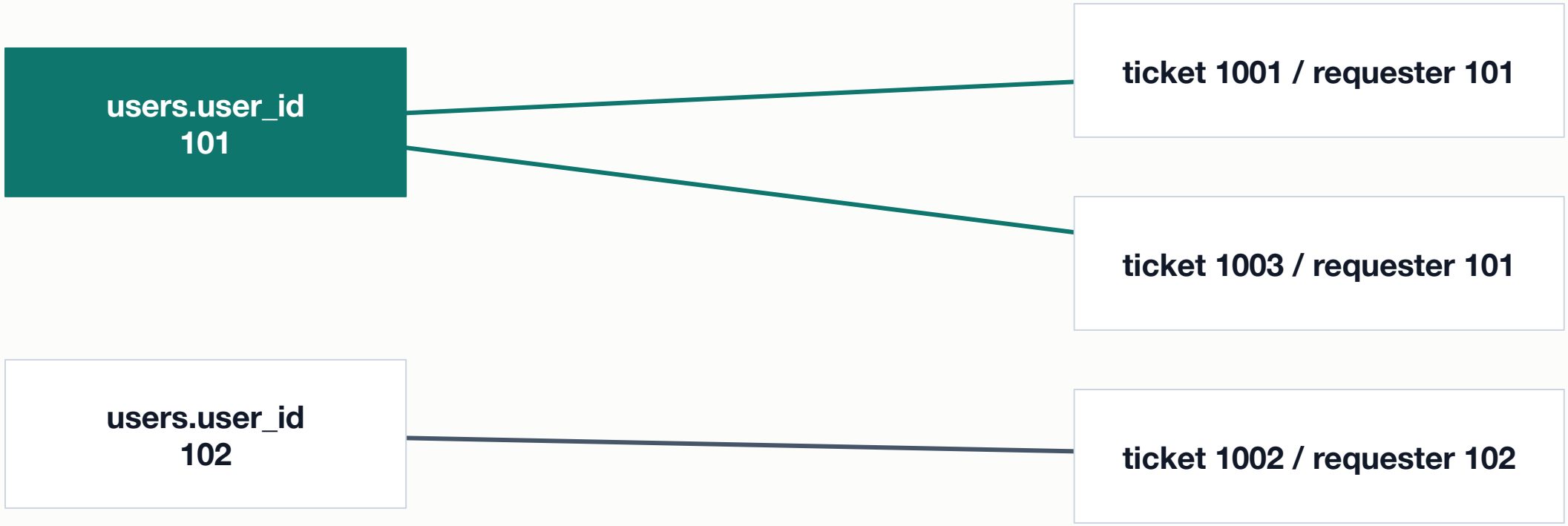
Sorting and limiting require a deterministic question

```
SELECT ticket_id, opened_at, priority
FROM tickets
WHERE status IN ('new', 'open', 'in_progress')
ORDER BY opened_at DESC, ticket_id DESC
LIMIT 3;
```

Meaning

- Filter active tickets first.
- Sort newest to oldest.
- Use ticket_id to break ties.

An inner join keeps matching tuple pairs



Result grain: one row per matching user-ticket pair, which is one row per ticket in this relationship.

Write the relationship in ON, then filter the result in WHERE

```
SELECT t.ticket_id, t.subject, u.display_name
FROM tickets AS t
JOIN users AS u
  ON u.user_id = t.requester_id
WHERE t.status = 'open'
ORDER BY t.ticket_id;
```

Read it in this order

- Source tickets and users
- Matching identity condition
- Open-ticket filter and output grain

LEFT JOIN preserves the left side and marks missing matches

INNER JOIN

- Keeps only matching pairs
- Drops a ticket with no matching assignee
- Useful when the relationship must exist for the question

LEFT JOIN

- Keeps every left-side ticket
- Produces NULL right-side attributes when unmatched
- Useful for finding unassigned or missing relationships

One-to-many joins can multiply rows without being wrong

Count the entity you mean, not whatever rows the join produced.

TICKET

One ticket may have many event tuples.

JOIN ROW

One output row can represent one ticket-event pair.

COUNT

Use the correct grain, grouping, or distinct identity.

GROUP BY changes grain; HAVING filters groups

```
SELECT status, count(*) AS ticket_count
FROM tickets
WHERE opened_at >= DATE '2026-02-01'
GROUP BY status
HAVING count(*) >= 2
ORDER BY ticket_count DESC, status;
```

Result grain

- WHERE filters source ticket rows.
- GROUP BY makes one row per status.
- HAVING keeps qualifying status groups.

Subqueries and CTEs can name intermediate reasoning

Subquery

- Nested inside another statement
- Useful for membership, existence, or a derived relation
- Correlated forms may run conceptually per outer row

Common table expression

- Names an intermediate result with WITH
- Can make multi-step grain changes visible
- Does not automatically guarantee materialization or speed

Set operations combine compatible result shapes

Operator	Membership	Duplicates	Typical question
UNION	rows from A or B	removes duplicates	all identifiers appearing in either result
UNION ALL	rows from A then B	keeps duplicates	append compatible event streams
INTERSECT	rows in both	distinct by default	identifiers meeting two independent conditions
EXCEPT	rows in A but not B	distinct by default	users with no matching membership in B

Test a data change inside a transaction before keeping it

```
BEGIN;

UPDATE tickets
SET priority = 'high'
WHERE ticket_id = 1002;

SELECT ticket_id, priority
FROM tickets
WHERE ticket_id = 1002;

ROLLBACK;
```

Safe rehearsal

- Use a narrow predicate and inspect affected rows.
- Verify the changed value inside the transaction.
- Rollback returns the fixture to its starting state.

Verification tests whether the result answers the question

1. Restate result grain and inspect stable identifiers or group keys.
2. Compare row count with a hand prediction or a simpler query.
3. Include edge cases: NULL, unmatched relationships, duplicates, and empty groups.
4. Use a second reasoning path and state what both checks still do not prove.

Lab 1: Complete the SQL query ladder

Translate increasingly complex relational questions into SQL and verify each result.

- Write grain notes before filters, projections, ordering, and limits.
- Predict stable identifiers or counts before running each query.
- Keep the SQL and a short verification note for the assigned checkpoint.

SUBMIT ONE THING / one SQL file with grain and verification notes

Lab 2: Join, summarize, and rehearse a safe change

Use relationships, grouping, a second reasoning path, and transaction-guarded DML.

- Build one inner or left join and account for every repeated or missing row.
- Create one grouped result with a correct group grain and filter location.
- Test one narrow UPDATE or DELETE, verify it, and roll it back.

SUBMIT ONE THING / one SQL evidence record

SQL fluency is recoverable when the result has a model

Do not trust syntax alone. Predict the relation, execute the query, and verify its meaning.

- What does one output row represent?
- Why do rows repeat or disappear?
- Which check could prove your interpretation wrong?